

Companion Notebook — *The Unified Codex: Algebraic Collapse and the Geometric Inversion of SHA-256*

This notebook is a **companion** to the paper, not a replacement for it. It focuses on the executable objects that can be checked directly from arithmetic, recurrence, and bit-level transport.

Scope of this companion

- portable notebook structure
- no hard-coded directories
- optional exports only
- teaching-first flow
- **no Z3 / SMT / SAT solver path**

The emphasis here is on the die equation, the NOP backbone, the carry channel, the Sziklai coupling, pure arithmetic 7/8-round reversal, and the GF(2) schedule rank structure.

```
In [1]: %pip install -q numpy pandas plotly sympy
```

WARNING: The directory '/home/oai/.cache/pip' or its parent directory is not owned or is not writable by the current user. The cache has been disabled. Check the permissions and owner of that directory. If executing pip with sudo, you should use sudo's -H flag.

Note: you may need to restart the kernel to use updated packages.

0. Minimal reference objects

The paper formalizes SHA-256 as a 64-cell die with state vector

$$[x_r = [a_r, b_r, c_r, d_r, e_r, f_r, g_r, h_r] \in (\mathbb{Z}/2^{32}\mathbb{Z})^8,]$$

and round update

$$[x_{r+1} = P x_r + u_a(T1_r + T2_r) + u_e, T1_r.]$$

In this companion, the die is treated as a **teaching object**: we verify what can be measured directly, then keep the paper nearby for the larger interpretive frame.

```
In [2]: from pathlib import Path
import os
import math
import time
import random
import statistics
from decimal import Decimal, getcontext

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

# Portable notebook configuration
EXPORT_ARTIFACTS = False
OUTPUT_DIR = Path.cwd() / "unified_codex_outputs"
if EXPORT_ARTIFACTS:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)

random.seed(42)
MASK32 = 0xFFFFFFFF
MOD = 1 << 32

H0 = [
    0x6a09e667, 0xbb67ae85, 0x3c6ef372, 0xa54ff53a,
    0x510e527f, 0x9b05688c, 0x1f83d9ab, 0x5be0cd19,
]

K64 = [
    0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfe, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6ff3,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90bffffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2
]
```

```

def rotr(x, n):
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def Sigma0(x):
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x):
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x):
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def sigma1(x):
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def ch(e, f, g):
    return (e & f) ^ ((~e) & g) & MASK32

def maj(a, b, c):
    return (a & b) ^ (a & c) ^ (b & c)

def add32(*xs):
    return sum(xs) & MASK32

def hw(x):
    return int(x).bit_count()

def pad_single_block(message: bytes) -> bytes:
    if len(message) > 55:
        raise ValueError("This companion notebook uses single-block messages only (<= 55 bytes).")
    bit_len = len(message) * 8
    padded = message + b"\x80"
    while len(padded) % 64 != 56:
        padded += b"\x00"
    padded += bit_len.to_bytes(8, "big")
    return padded

def words_from_message(message: bytes):
    padded = pad_single_block(message)
    return [int.from_bytes(padded[i:i+4], "big") for i in range(0, 64, 4)]

```

```

def msg_schedule(words16):
    W = list(words16) + [0] * 48
    for r in range(16, 64):
        W[r] = add32(sigma1(W[r-2]), W[r-7], sigma0(W[r-15]), W[r-16])
    return W

def exact_carry_word(x, y):
    carry_word = 0
    c = 0
    for i in range(32):
        xi = (x >> i) & 1
        yi = (y >> i) & 1
        c = (xi & yi) | (xi & c) | (yi & c)
        if c:
            carry_word |= (1 << i)
    return carry_word

def carry_out_count(*xs):
    return (sum(xs) >> 32)

def compress_trace(words16, rounds=64):
    W = msg_schedule(words16)
    a, b, c, d, e, f, g, h = H0
    rows = []
    for r in range(rounds):
        t1 = add32(h, Sigma1(e), ch(e, f, g), K64[r], W[r])
        t2 = add32(Sigma0(a), maj(a, b, c))
        carry_t1_word = 0
        partial = [h, Sigma1(e), ch(e, f, g), K64[r], W[r]]
        acc = 0
        for item in partial:
            carry_t1_word |= exact_carry_word(acc, item)
            acc = add32(acc, item)
        carry_t2_word = exact_carry_word(Sigma0(a), maj(a, b, c))
        a_next = add32(t1, t2)
        e_next = add32(d, t1)
        rows.append(
            {
                "round": r,
                "a": a, "b": b, "c": c, "d": d, "e": e, "f": f, "g": g, "h": h,
                "W": W[r], "T1": t1, "T2": t2,
                "carry_T1_count": carry_out_count(h, Sigma1(e), ch(e, f, g), K64[r], W[r]),
            }
        )

```

```

        "carry_T2_count": carry_out_count(Sigma0(a), maj(a, b, c)),
        "carry_T1_word": carry_t1_word,
        "carry_T2_word": carry_t2_word,
        "carry_T1_hw": hw(carry_t1_word),
        "carry_T2_hw": hw(carry_t2_word),
        "a_next": a_next,
        "e_next": e_next,
        "sziklai_residual": ((a_next - e_next) - (t2 - d)) & MASK32,
    }
)
a, b, c, d, e, f, g, h = a_next, a, b, c, e_next, e, f, g
return pd.DataFrame(rows)

def primes(n):
    out = []
    x = 2
    while len(out) < n:
        isprime = True
        for p in out:
            if p * p > x:
                break
            if x % p == 0:
                isprime = False
                break
        if isprime:
            out.append(x)
        x += 1
    return out

def cube_root_fraction_word(p):
    getcontext().prec = 100
    x = Decimal(p) ** (Decimal(1) / Decimal(3))
    frac = x - int(x)
    return int(frac * (1 << 32))

def schedule_rank_derived():
    # rows correspond to the 1536 derived bits in W[16..63]
    rows = [0] * (48 * 32)
    for col in range(512):
        word = col // 32
        bit = col % 32
        seed = [0] * 16

```

```

    seed[word] = 1 << bit
    W = msg_schedule(seed)
    for r in range(48):
        x = W[16 + r]
        base = r * 32
        while x:
            lsb = x & -x
            b = lsb.bit_length() - 1
            rows[base + b] |= (1 << col)
            x ^= lsb
    rank = gf2_rank_rows(rows, 512)
    return rank, 512 - rank, rows

def gf2_rank_rows(rows, ncols):
    rows = rows[:]
    rank = 0
    for col in range(ncols - 1, -1, -1):
        mask = 1 << col
        pivot = None
        for r in range(rank, len(rows)):
            if rows[r] & mask:
                pivot = r
                break
        if pivot is None:
            continue
        rows[rank], rows[pivot] = rows[pivot], rows[rank]
        piv = rows[rank]
        for r in range(len(rows)):
            if r != rank and (rows[r] & mask):
                rows[r] ^= piv
        rank += 1
    if rank == ncols:
        break
    return rank

# A-Mark9 pure arithmetic reversal helpers (no search / no solver path)
S0_ = Sigma0
S1_ = Sigma1
Ch = ch
Mj = maj

def Bv(i, av):

```

```

    return H0[1] if i == 0 else av[i - 1]

def Cv(i, av):
    return {0: H0[2], 1: H0[1]}.get(i, av[i - 2])

def T2x(i, av):
    return add32(S0_(av[i]), Mj(av[i], Bv(i, av), Cv(i, av)))

def Dv(i, av):
    return {0: H0[3], 1: H0[2], 2: H0[1]}.get(i, av[i - 3])

def sha_forward(W16, R):
    W = msg_schedule(W16)
    a, b, c, d, e, f, g, h = H0
    hist = [[a, b, c, d, e, f, g, h]]
    for i in range(R):
        t1 = add32(h, S1_(e), Ch(e, f, g), K64[i], W[i])
        t2 = add32(S0_(a), Mj(a, b, c))
        h, g, f = g, f, e
        e = add32(d, t1)
        d, c, b = c, b, a
        a = add32(t1, t2)
        hist.append([a, b, c, d, e, f, g, h])
    return hist

def invert_7(state7):
    av = [None] * 8
    ev = [None] * 8
    av[7], av[6], av[5], av[4] = state7[0], state7[1], state7[2], state7[3]
    ev[7], ev[6], ev[5], ev[4] = state7[4], state7[5], state7[6], state7[7]
    av[3] = (T2x(6, av) - (av[7] - ev[7])) % MOD
    av[2] = (T2x(5, av) - (av[6] - ev[6])) % MOD
    av[1] = (T2x(4, av) - (av[5] - ev[5])) % MOD
    av[0] = H0[0]
    ev[0] = H0[4]
    for i in range(7):
        ev[i + 1] = (av[i + 1] - T2x(i, av) + Dv(i, av)) % MOD
    d = [Dv(i, av) for i in range(7)]
    T1 = [(ev[i + 1] - d[i]) % MOD for i in range(7)]
    h_hist = [H0[7], H0[6], H0[5]] + [ev[i - 3] for i in range(3, 7)]
    f_hist = [H0[5]] + [ev[i - 1] for i in range(1, 7)]
    g_hist = [H0[6], H0[5]] + [ev[i - 2] for i in range(2, 7)]

```

```

    return [(T1[i] - h_hist[i] - S1(ev[i]) - Ch(ev[i], f_hist[i], g_hist[i]) - K64[i]) % MOD for i in range(7)]

def invert_8(state8):
    av = [None] * 9
    ev = [None] * 9
    av[8], av[7], av[6], av[5] = state8[0], state8[1], state8[2], state8[3]
    ev[8], ev[7], ev[6], ev[5] = state8[4], state8[5], state8[6], state8[7]
    av[4] = (T2x(7, av) - (av[8] - ev[8])) % MOD
    av[3] = (T2x(6, av) - (av[7] - ev[7])) % MOD
    av[2] = (T2x(5, av) - (av[6] - ev[6])) % MOD
    av[1] = (T2x(4, av) - (av[5] - ev[5])) % MOD
    av[0] = H0[0]
    ev[0] = H0[4]
    for i in range(8):
        ev[i + 1] = (av[i + 1] - T2x(i, av) + Dv(i, av)) % MOD
    d = [Dv(i, av) for i in range(8)]
    T1 = [(ev[i + 1] - d[i]) % MOD for i in range(8)]
    h_hist = [H0[7], H0[6], H0[5]] + [ev[i - 3] for i in range(3, 8)]
    f_hist = [H0[5]] + [ev[i - 1] for i in range(1, 8)]
    g_hist = [H0[6], H0[5]] + [ev[i - 2] for i in range(2, 8)]
    return [(T1[i] - h_hist[i] - S1(ev[i]) - Ch(ev[i], f_hist[i], g_hist[i]) - K64[i]) % MOD for i in range(8)]

```

```

In [3]: component_rows = [
    {"Object": "State vector", "Symbol": "$x_r$", "Role": "Eight 32-bit working lanes"},
    {"Object": "Ground fold", "Symbol": "$T2_r$", "Role": "North-bridge fold from (a,b,c)"},
    {"Object": "Live wire", "Symbol": "$T1_r$", "Role": "South-bridge injection from (e,f,g,h) plus K_r and W_r"},
    {"Object": "Clock rail", "Symbol": "$K_r$", "Role": "Fixed round constant"},
    {"Object": "Payload rail", "Symbol": "$W_r$", "Role": "Message schedule word"},
    {"Object": "Shift map", "Symbol": "$P$", "Role": "Register transport between adjacent rounds"},
]
component_df = pd.DataFrame(component_rows)
component_df

```

Out[3]:

	Object	Symbol	Role
0	State vector	x_r	Eight 32-bit working lanes
1	Ground fold	$T2_r$	North-bridge fold from (a,b,c)
2	Live wire	$T1_r$	South-bridge injection from (e,f,g,h) plus K_r ...
3	Clock rail	K_r	Fixed round constant
4	Payload rail	W_r	Message schedule word
5	Shift map	P	Register transport between adjacent rounds

1. Die geometry at one glance

This diagram is intentionally compact. It is not a transistor schematic. It is a teaching picture for the two active injections of each round.

```
In [4]: fig_die = go.Figure()

nodes = {
    "a,b,c": (0.10, 0.78),
    "e,f,g,h": (0.10, 0.28),
    "K[i], W[i]": (0.10, 0.08),
    "T2": (0.42, 0.78),
    "T1": (0.42, 0.22),
    "a[i+1]": (0.80, 0.78),
    "e[i+1]": (0.80, 0.22),
}
edges = [
    ("a,b,c", "T2"),
    ("e,f,g,h", "T1"),
    ("K[i], W[i]", "T1"),
    ("T2", "a[i+1]"),
    ("T1", "a[i+1]"),
    ("T1", "e[i+1]"),
]
for src, dst in edges:
    x0, y0 = nodes[src]
```

```

x1, y1 = nodes[dst]
fig_die.add_annotation(
    x=x1, y=y1, ax=x0, ay=y0, xref="x", yref="y", axref="x", ayref="y",
    showarrow=True, arrowhead=3, arrowsize=1, arrowwidth=1.5
)
for name, (x, y) in nodes.items():
    fig_die.add_trace(go.Scatter(
        x=[x], y=[y], mode="markers+text", text=[name], textposition="middle center",
        marker=dict(size=48, opacity=0.8),
        hoverinfo="skip", showlegend=False
    ))
fig_die.update_layout(
    title="Single-round die picture: two nonlinear injections into the next state",
    xaxis=dict(visible=False, range=[0, 1]),
    yaxis=dict(visible=False, range=[0, 1]),
    height=420,
    margin=dict(l=20, r=20, t=60, b=20),
)
fig_die.show()

```

2. Ground fold and round-zero displacement

The paper isolates the universal ground fold

$$T2_0 = \Sigma_0(a_0) + \operatorname{Maj}(a_0, b_0, c_0) = 0x08909ae5.$$

It also states that at round zero the message enters as a pure displacement against the fixed ground plane. The cleanest way to see that is to compare a real run against the NOP backbone.

```

In [5]: trace_nop = compress_trace([0] * 16)
        trace_abc = compress_trace(words_from_message(b"abc"))

T2_0 = int(trace_nop.loc[0, "T2"])
W0_abc = int(trace_abc.loc[0, "W"])
delta_T1_0 = (int(trace_abc.loc[0, "T1"]) - int(trace_nop.loc[0, "T1"])) & MASK32
delta_a1_0 = (int(trace_abc.loc[1, "a"]) - int(trace_nop.loc[1, "a"])) & MASK32
delta_e1_0 = (int(trace_abc.loc[1, "e"]) - int(trace_nop.loc[1, "e"])) & MASK32

pd.DataFrame([
    {"quantity": "T2_0", "value_hex": hex(T2_0)},
    {"quantity": "W0 for b'abc'", "value_hex": hex(W0_abc)},
])

```

```

{"quantity": "T1_0 - T1_0^(NOP)", "value_hex": hex(delta_T1_0)},
{"quantity": "a_1 - a_1^(NOP)", "value_hex": hex(delta_a1_0)},
{"quantity": "e_1 - e_1^(NOP)", "value_hex": hex(delta_e1_0)},
])

```

```

Out[5]:

```

	quantity	value_hex
0	T2_0	0x8909ae5
1	W0 for b'abc'	0x61626380
2	T1_0 - T1_0^(NOP)	0x61626380
3	a_1 - a_1^(NOP)	0x61626380
4	e_1 - e_1^(NOP)	0x61626380

2.1 Perturbation heatmap

A companion notebook should show **where the message shows up first**. Here the color is the lane-wise Hamming distance between the NOP backbone and the real run for the message `b"abc"`.

```

In [6]: lanes = list("abcdefgh")
delta_rows = []
for r in range(64):
    for lane in lanes:
        delta_rows.append({
            "round": r,
            "lane": lane,
            "delta_hw": hw(int(trace_abc.loc[r, lane]) ^ int(trace_nop.loc[r, lane]))
        })
delta_df = pd.DataFrame(delta_rows)

fig_delta = px.imshow(
    delta_df.pivot(index="lane", columns="round", values="delta_hw"),
    aspect="auto",
    labels=dict(color="Hamming distance"),
    title="Lane-wise displacement relative to the NOP backbone"
)

```

```
fig_delta.update_layout(height=420)
fig_delta.show()
```

3. Carry channel geometry

The A-Mark9 material pushes the carry channel into the foreground. The notebook keeps that, but in a compact form: NOP signature, per-round carry weights, and distance-to-NOP over random single-block messages.

```
In [7]: t2_signature_bits = "".join(str(int(v)) for v in trace_nop["carry_T2_count"].tolist())
t2_signature_weight = t2_signature_bits.count("1")
pd.DataFrame([{"T2 carry signature (NOP)": t2_signature_bits,
               "weight": t2_signature_weight,
               "length_bits": len(t2_signature_bits),
               }])
```

```
Out[7]:
```

	T2 carry signature (NOP)	weight	length_bits
0	1101111000011010010101000101011010001101011000...	34	64

```
In [8]: fig_carry = go.Figure()
fig_carry.add_trace(go.Scatter(
    x=trace_nop["round"], y=trace_nop["carry_T1_hw"],
    mode="lines+markers", name="NOP carry_T1 hw"
))
fig_carry.add_trace(go.Scatter(
    x=trace_nop["round"], y=trace_nop["carry_T2_hw"],
    mode="lines+markers", name="NOP carry_T2 hw"
))
fig_carry.add_trace(go.Scatter(
    x=trace_abc["round"], y=trace_abc["carry_T1_hw"],
    mode="lines", name="abc carry_T1 hw"
))
fig_carry.add_trace(go.Scatter(
    x=trace_abc["round"], y=trace_abc["carry_T2_hw"],
    mode="lines", name="abc carry_T2 hw"
))
fig_carry.update_layout(
```

```

    title="Carry-word Hamming weights by round",
    xaxis_title="round",
    yaxis_title="Hamming weight",
    height=460
)
fig_carry.show()

```

3.1 Distance from the NOP carry signature

This is a compact way to quantify how quickly the carry shape separates from the pure-constant backbone.

```

In [9]: def random_one_block_message():
        n = random.randint(1, 55)
        return os.urandom(n)

nop_sig = np.array(trace_nop["carry_T2_count"].astype(int).tolist(), dtype=np.uint8)
distances = []
for _ in range(200):
    tr = compress_trace(words_from_message(random_one_block_message()))
    sig = np.array(tr["carry_T2_count"].astype(int).tolist(), dtype=np.uint8)
    distances.append(int(np.sum(sig ^ nop_sig)))

distance_df = pd.DataFrame({"hamming_distance_to_NOP_T2_signature": distances})
distance_df.describe()

```

Out[9]: **hamming_distance_to_NOP_T2_signature**

count	200.000000
mean	31.605000
std	3.873499
min	21.000000
25%	29.000000
50%	31.000000
75%	34.250000
max	43.000000

```
In [10]: fig_dist = px.histogram(
    distance_df,
    x="hamming_distance_to_NOP_T2_signature",
    nbins=20,
    title="Distribution of Hamming distance to the NOP T2 carry signature"
)
fig_dist.add_vline(x=distance_df.iloc[:,0].mean(), line_dash="dash")
fig_dist.update_layout(height=420)
fig_dist.show()
```

4. Sziklai coupling identity

The paper uses the algebraic relation $[a_{i+1} - e_{i+1} \equiv T2_i - d_i \pmod{2^{32}}]$.

This notebook treats it as a direct executable invariant.

```
In [11]: check_rows = []
violations = 0
for msg_idx in range(500):
    tr = compress_trace(words_from_message(random_one_block_message()))
    residuals = tr["sziklai_residual"].astype(int)
    violations += int((residuals != 0).sum())
```

```

        check_rows.append({"message_index": msg_idx, "all_rounds_zero": bool((residuals == 0).all())})

sziklai_summary = pd.DataFrame([
    "messages_checked": 500,
    "rounds_per_message": 64,
    "total_rounds_checked": 500 * 64,
    "violations": violations,
    "all_messages_passed": violations == 0,
])
sziklai_summary

```

```

Out[11]:
  messages_checked  rounds_per_message  total_rounds_checked  violations  all_messages_passed
0                500                64                32000            0                True

```

5. Pure arithmetic 7/8-round reversal

This section uses the latest A-Mark9 arithmetic inversion path only.

There is **no solver path here**. No Z3, no SAT, no SMT. The object is purely the recurrence itself: for `R <= 8`, recover the leading schedule words directly from the terminal state.

```

In [12]: bench_rows = []
for R in [7, 8]:
    latencies = []
    ok = 0
    trials = 300
    for _ in range(trials):
        W16 = [random.randint(0, MOD - 1) for _ in range(16)]
        hist = sha_forward(W16, R)
        t0 = time.perf_counter()
        recovered = invert_7(hist[7]) if R == 7 else invert_8(hist[8])
        latencies.append((time.perf_counter() - t0) * 1000.0)
        width = 7 if R == 7 else 8
        ok += int(all(recovered[i] == W16[i] for i in range(width)))
    bench_rows.append({
        "rounds": R,
        "trials": trials,
        "exact_recoveries": ok,
    })

```

```

        "recovery_rate": ok / trials,
        "mean_latency_ms": statistics.mean(latencies),
        "median_latency_ms": statistics.median(latencies),
        "max_latency_ms": max(latencies),
    })
    bench_df = pd.DataFrame(bench_rows)
    bench_df

```

```

Out[12]:

```

	rounds	trials	exact_recoveries	recovery_rate	mean_latency_ms	median_latency_ms	max_latency_ms
0	7	300	300	1.0	0.032697	0.029707	0.149287
1	8	300	300	1.0	0.036259	0.034597	0.137626

```

In [13]: fig_rev = go.Figure()
fig_rev.add_trace(go.Bar(
    x=bench_df["rounds"].astype(str),
    y=bench_df["recovery_rate"],
    text=[f"{v:.3f}" for v in bench_df["recovery_rate"]],
    textposition="outside",
    name="recovery rate"
))
fig_rev.update_layout(
    title="Pure arithmetic reversal benchmark",
    xaxis_title="round horizon",
    yaxis_title="exact recovery rate",
    yaxis_range=[0, 1.08],
    height=420
)
fig_rev.show()

```

6. GF(2) rank of the derived message schedule

The paper states that the schedule expansion is injective at the linear GF(2) level. The compact check here is the rank of the map from 512 input bits to the 1536 **derived** bits in `W[16..63]`.

```

In [14]: rank_derived, nullity_derived, _derived_rows = schedule_rank_derived()
pd.DataFrame([
    "domain_bits": 512,

```

```

    "codomain_bits_derived_only": 48 * 32,
    "GF(2) rank": rank_derived,
    "nullity": nullity_derived,
    "injective": nullity_derived == 0,
  })

```

```

Out[14]:

```

	domain_bits	codomain_bits_derived_only	GF(2) rank	nullity	injective
0	512	1536	512	0	True

7. Prime-root rails

The round constants are the fractional parts of the cube roots of the first 64 primes, scaled into 32-bit words. A short audit is useful because it keeps the constants anchored to a direct arithmetic construction.

```

In [15]: prime_list = primes(64)
prime_audit = pd.DataFrame({
    "index": np.arange(64),
    "prime": prime_list,
    "K_hex": [hex(k) for k in K64],
    "K_fractional": [k / (1 << 32) for k in K64],
    "expected_hex": [hex(cube_root_fraction_word(p)) for p in prime_list],
})
prime_audit["match"] = prime_audit["K_hex"] == prime_audit["expected_hex"]
prime_audit.head(10)

```

Out[15]:

	index	prime	K_hex	K_fractional	expected_hex	match
0	0	2	0x428a2f98	0.259921	0x428a2f98	True
1	1	3	0x71374491	0.442250	0x71374491	True
2	2	5	0xb5c0fbcf	0.709976	0xb5c0fbcf	True
3	3	7	0xe9b5dba5	0.912931	0xe9b5dba5	True
4	4	11	0x3956c25b	0.223980	0x3956c25b	True
5	5	13	0x59f111f1	0.351335	0x59f111f1	True
6	6	17	0x923f82a4	0.571282	0x923f82a4	True
7	7	19	0xab1c5ed5	0.668402	0xab1c5ed5	True
8	8	23	0xd807aa98	0.843867	0xd807aa98	True
9	9	29	0x12835b01	0.072317	0x12835b01	True

```
In [16]: fig_prime = px.scatter(
    prime_audit,
    x="index",
    y="K_fractional",
    hover_data=["prime", "K_hex", "expected_hex", "match"],
    title="SHA-256 round constants as cube-root fractional rails"
)
fig_prime.update_layout(height=430)
fig_prime.show()
```

8. Optional export

By default, this notebook writes nothing. If you want a few clean artifacts for a paper folder or teaching handout, turn `EXPORT_ARTIFACTS = True` in the imports cell and rerun.

```
In [17]: if EXPORT_ARTIFACTS:
    component_df.to_csv(OUTPUT_DIR / "die_reference_table.csv", index=False)
    bench_df.to_csv(OUTPUT_DIR / "arithmetic_reversal_benchmark.csv", index=False)
```

```
prime_audit.to_csv(OUTPUT_DIR / "prime_root_audit.csv", index=False)
distance_df.to_csv(OUTPUT_DIR / "carry_distance_histogram_data.csv", index=False)
print(f"Exports written to: {OUTPUT_DIR.resolve()}")
else:
    print("EXPORT_ARTIFACTS is False – no files written.")
```

EXPORT_ARTIFACTS is False – no files written.

9. Exercises and next checks

1. Replace `b"abc"` with another single-block message and rerun the perturbation heatmap.
2. Increase the random sample count in the carry-distance and coupling sections.
3. Change the reversal benchmark from 300 to 1000 trials and compare the latency distribution.
4. Extend the derived-schedule rank cell to include the original 512 bits if you want the full 2048-bit schedule view.
5. Add your own arithmetic-only coordinate next — but keep the notebook in the same companion style: no embedded paper, no hard-coded paths, no dump-for-dump's-sake sections.